

Local Functions in C# 7 with Examples

In this article, I am going to discuss the **Local Functions in C#** with Examples that are introduced as part of C# 7. Please read our previous article where we discussed how to [Splitting Tuples in C# 7](#) with Examples. The Local Functions means a function is declared and defined inside another function.

What are Local Functions in C#?

The **Local Functions in C#** are the special kind of inner function or you can say sub-function or function within a function that can be declared and defined by the parent function. These methods or functions are private methods for their containing type and are only called by their parent method.

Why do we need Local Functions in C#?

If you want to execute some piece of code multiple times within a method then you can put those codes as an inner function or you can say local function within that method. Then call that local function whenever required from the parent method. Some of the examples where we can create local functions are as follows

1. Small helper functions are to be used several times within the main or parent method.
2. Parameter validation functions for any iterators or asynchronous methods.
3. An alternate to recursive functions as local function comparatively takes less memory due to the reduced call stack.

Example to Understand Local Functions in C# 7

Let's understand Local Functions in C# 7 with one example. Please have a look at the below code. As you can see, within the Main method we are defining two inner methods i.e. Sum and Difference. The Main method is called the Parent Method and the Sum and Difference methods are called Local Functions or Methods. You can access Sum and Difference method in the context of the Main method only.

```
using System;
class Program
{
    //Main Method is Parent Method
    static void Main()
    {
        int a = 10, b = 5;
        //Calling the Local Function
        int sum = Sum(a, b);
        //Calling the Local Function
        int difference = Difference(a, b);
        Console.WriteLine($"The Sum of {a} and {b} is {sum}");
        Console.WriteLine($"The Difference of {a} and {b} is {difference}");
        //Local Function
        //Access Specifier not Allowed
        int Sum(int x, int y)
        {
            return x + y;
        }
    }
}
```

```

    }
    //Local Function
    //Access Specifier not Allowed
    int Difference(int x, int y)
    {
        return x - y;
    }
    Console.WriteLine("Press any key to exit.");
    Console.ReadKey();
}
}

```

When we run the application, it will give us the following output.

```

The Sum of 10 and 5 is 15
The Difference of 10 and 5 is 5
Press any key to exit.

```

As you can see in the above example, we have created two nested functions “**Sum**” and “**Difference**”. These two local functions can be called from anywhere by the parent’s main function only.

Points to Remember while working with Local Functions in C# 7:

The following points you need to keep in mind while working with the Local Functions.

1. You can not overload a Local Function in C#
2. The Accessibility modifiers such as public, private, and protected are not allowed.
3. The compiler will issue a warning if the local function is not used by the parent function as there is no meaning of defining a local function in C# if it is not being used by the parent method.
4. All variables in the enclosing scope, including local variables, can be accessed

Real-Time Example of Local Functions in C# 7:

Let us understand the use of Local Functions with one real-time example. The Parameter Validation scenario. In the following example, the **IsRequestValid** local function is used to validate the parameters of the **AddEmployee** function.

```

using System;
using System.Text;
namespace LocalFunctionDemo
{
    class Program
    {
        static void Main()
        {
            Employee employee1 = new Employee()
            {
                Id = 1001,

```

```

        Name = "Pranaya",
        Gender = "Male",
        Salary = 1000,
        Department = "IT"
    };
    bool IsInserted = AddEmployee(employee1);
    Console.WriteLine($"Is Employee with id 1001 inserted: {IsInserted}");
    Employee employee2 = new Employee()
    {
        Id = 1001,
        Name = "Pranaya",
        Department = "IT"
    };
    IsInserted = AddEmployee(employee2);
    Console.WriteLine($"Is Employee with id 1002 inserted: {IsInserted}");
    Console.WriteLine("Press any key to exit.");
    Console.ReadKey();
}
//AddEmployee is the Parent Function
public static bool AddEmployee(Employee request)
{
    var validationResult = IsRequestValid();
    if (validationResult.IsValid == false)
    {
        Console.WriteLine($"Error: {validationResult.ErrorMessage}");
        return false;
    }
    // Some code for inserting the Employee in database.
    return true;
    //Using Tuple to return two values from IsRequestValid Local Function
    (bool IsValid, string ErrorMessage) IsRequestValid()
    {
        if (request == null)
        {
            throw new ArgumentNullException(nameof(request), $"The {
nameof(request)} can not be null.");
        }
        StringBuilder SB = new StringBuilder();
        if (string.IsNullOrEmpty(request.Name))
        {
            SB.AppendLine($"The {nameof(request)}'s {nameof(request.Name)}
property can not be empty.");
        }
        if (string.IsNullOrEmpty(request.Gender))
        {
            SB.AppendLine($"The {nameof(request)}'s {nameof(request.Gender)}
property can not be empty.");
        }
        if (string.IsNullOrEmpty(request.Department))
        {

```

```

        SB.AppendLine($"The {nameof(request)}'s {nameof(request.Department)} property can not be empty.");
    }
    if (request.Id <= 0)
    {
        SB.AppendLine($"The {nameof(request)}'s {nameof(request.Id)} can not be less than zero.");
    }
    if (request.Salary <= 0)
    {
        SB.AppendLine($"The {nameof(request)}'s {nameof(request.Salary)} property can not be less than zero.");
    }
    if (!string.IsNullOrEmpty(Convert.ToString(SB)))
    {
        var errorMessage = SB.ToString();
        return (IsValid: false, ErrorMessage: errorMessage);
    }
    return (IsValid: true, ErrorMessage: string.Empty);
}
}
}
}
}
public class Employee
{
    public long Id { get; set; }
    public string Name { get; set; }
    public string Gender { get; set; }
    public double Salary { get; set; }
    public string Department { get; set; }
}
}
}

```

When we run the application, it will give us the following output.

```

Is Employee with id 1001 inserted: True
errorMessage : The request's Gender property can not be empty.
The request's Salary property can not be less than zero.
Is Employee with id 1002 inserted: False
Press any key to exit.

```

In the next article, I am going to discuss [Ref Returns and Ref Locals in C# 7](#) with Examples. Here, in this article, I try to explain the **Local Functions in C# 7** with Examples. I hope this article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about this article.